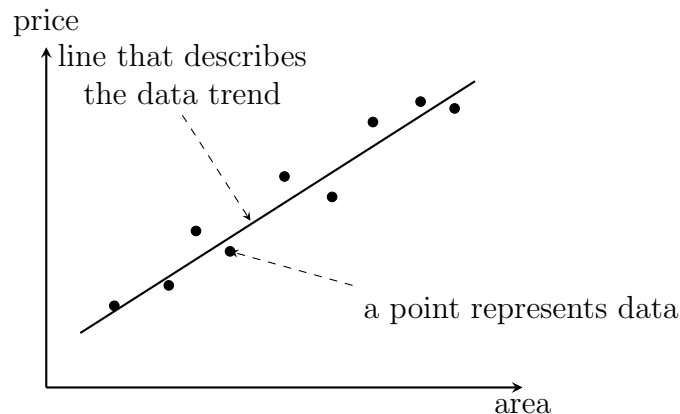


Lecture 1. Linear Regression

Linear regression predicts a numeric value based on given features by fitting a line that lies as close as possible to the data. For example: apartment price as a function of area:



Thus, the line helps predict the approximate price of a subsequent apartment.

Now we will learn how to train the model, i.e., how to find the equation of the line that captures the underlying pattern in the data. As we know from mathematics, a line (hyperplane) in an n -dimensional space is given by the equation:

$$y = w_0 + w_1x_1 + \cdots + w_nx_n$$

The objective of linear regression is to find the coefficients $w_0, \dots, w_n$ such that the target values y (e.g., housing prices) are as close to the line as possible.

The input data in tabular form is structured as follows:

No.	x_1	x_2	$\dots$	y
1				
2				
$\dots$				
m				

where x_i denote features (area, floor number, etc.), and y denotes the target value (e.g., price).

On the one hand, the weights can easily be found by solving the system of equations $X\vec{w} = \vec{y}$; multiplying by X^T :

$$\begin{aligned}\frac{X^T X w}{A} &= \frac{X^T y}{b} \\ A\vec{w} &= b \\ \vec{w} &= A^{-1}b\end{aligned}$$

However, this is computationally expensive (it takes a long time to compute). There is a method that is several times faster: the conjugate gradient method.

It initializes $w_0, \dots, w_n$ with random numbers, and then refines them at each step:

$$w_i^k = w_i^{k-1} - \eta \cdot \nabla L(w^{k-1})$$

where k is the step number, η is the step size (learning rate), and L is the gradient of the error at step $k - 1$.

The gradient formula for step k is given by:

$$\nabla L(w^k) = \left(\frac{\partial L}{\partial w_1^k}, \dots, \frac{\partial L}{\partial w_n^k} \right)^T$$

where L is the magnitude of the error (recomputed at each step). There are several formulas for it, which we will discuss later.

First, I want to discuss sparse-cg, which eliminates the need to manually set the step size by computing its optimal value at each iteration.

Thus, linear regression using sparse-cg uses the following update rule for the weights:

$$w^{k+1} = w^k + \alpha_k p_k$$

where:

- w^k denotes the current weights (at step k);
- p_k is the search direction;
- α_k indicates how far to move along this direction;
- $r_k = b - Aw_k$ is the residual;
- $A = X^T X$ (for linear regression).

Now, I will describe in detail the linear regression algorithm using sparse-cg (since, according to research findings, this variant of gradient descent is the fastest and most optimal).

Step 1

From the dataset file, we construct the feature matrix X and the ground-truth target vector y :

$$X = \begin{pmatrix} x_{11} & x_{12} & \dots & x_{1n} \\ x_{21} & x_{22} & \dots & x_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ x_{m1} & x_{m2} & \dots & x_{mn} \end{pmatrix}, \quad y = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{pmatrix}$$

where m is the number of samples and n is the number of features.

The prediction formula for $\hat{y}$ across all samples is given by:

$$\hat{y} = X \cdot w$$

and for an individual sample:

$$\hat{y}_i = w_1 x_{i1} + \dots + w_n x_{in}$$

Step 2

We define the loss function as:

$$L(w) = \sum_{i=1}^m (\hat{y}_i - y_i)^2$$

Step 3

For this error, the gradient formula is:

$$\nabla L(w) = 2X^T(Xw - y)$$

Step 4

We cast the problem into the linear system form $Aw = b$:

$$A = X^T X, \quad b = X^T y$$

This system will now be solved using the conjugate gradient method.

Step 5

We initialize the weights, for example, as:

$$w_0 = \begin{pmatrix} 0 \\ 0 \\ \vdots \\ 0 \end{pmatrix}$$

Step 6

We compute the initial residual:

$$r_0 = b - Aw_0$$

and the initial search direction:

$$p_0 = r_0$$

Step 7

At each iteration, we compute:

1. the step length:

$$\alpha_k = \frac{r_k^T \cdot r_k}{p_k^T \cdot X^T \cdot X \cdot p_k}$$

2. the updated weights:

$$w_{k+1} = w_k + \alpha_k p_k$$

3. the updated residual:

$$r_{k+1} = r_k - \alpha_k A p_k$$

4. the new search direction:

$$p_{k+1} = r_{k+1} + \beta_k \cdot p_k, \quad \text{where } \beta_k = \frac{r_{k+1}^T \cdot r_{k+1}}{r_k^T \cdot r_k}$$

We continue repeating this step while $\|r_k\| < \varepsilon$.

Step 8 (Testing Stage)

1. Using the resulting weights, we compute predictions for the test set:

$$\hat{y}_i = w_0 + w_1 x_{i1} + \cdots + w_n x_{in}$$

where:

- $\hat{y}_i$ is the prediction for the i -th sample;

- $w_0, \dots, w_n$ are the weights obtained during training;
- $x_{i1}, \dots, x_{in}$ are the features of the i -th sample.

2. We evaluate the error magnitude:

$$L(w) = \sum_{i=1}^m (y_i - \hat{y}_i)^2$$

This error formulation is called SSE (Sum of Squared Errors).

In Python, sparse-cg is invoked via Ridge in scikit-learn:

```
from sklearn.linear_model import Ridge

model = Ridge(alpha=0.0, solver='sparse_cg')
model.fit(X, y)
```

Here, `alpha = 0.0` indicates the absence of regularization, meaning that we are fitting ordinary linear regression.